UT2 · Uso de estilos

Apuntes

Diseño de Interfaces Web (0615) · 2.º DAW
IES La Hontanilla · Tarancón (Cuenca)

Curso 2026/2027

Índice general

1 Estilos en línea basados en etiquetas y en clases **3**

1.1 Separar contenido y presentación . . . 3
1.2 Las etiquetas HTML y su presentación por defecto . . . 3
1.3 Posibilidades de modificar las etiquetas HTML . . . 4
1.4 Atributos globales relacionados con los estilos . . . 4
1.5 Sintaxis de una regla CSS . . . 4
1.6 Estilos en línea: el atributo `style` . . . 5
1.7 Estilos basados en etiquetas: el selector de tipo . . . 5
1.8 Estilos basados en clases . . . 5
1.9 El selector de identificador . . . 6
1.10 Nombrar clases: buenas prácticas y BEM . . . 6

2 Crear y vincular hojas de estilo **7**

2.1 Hoja de estilos interna: el elemento `style` . . . 7
2.2 Selectores combinados . . . 7
2.3 Selectores de atributo . . . 8
2.4 Pseudoclases . . . 8
2.5 Pseudoelementos . . . 9
2.6 La cascada . . . 9
2.7 Especificidad . . . 9
2.8 Herencia . . . 10
2.9 Redefinir estilos . . . 11

3 Crear y vincular hojas de estilo en cascada externa **11**

3.1 La hoja externa y el elemento `link` . . . 11
3.2 Comparación de las tres formas de aplicar CSS . . . 12
3.3 Organización de una hoja global . . . 12
3.4 Hojas de estilo alternativas . . . 13
3.5 Propiedades personalizadas (variables CSS) . . . 14

4 Propiedades de los elementos **14**
4.1 Valores y unidades . . . 14
4.2 Color y fondos . . . 15
4.3 Texto y fuentes . . . 16
4.4 El modelo de caja . . . 16
4.5 Tipo de visualización: `display` . . . 17
4.6 Posicionamiento . . . 18

5 Maquetación con Flexbox y Grid **18**
5.1 Flexbox . . . 18
5.2 Grid . . . 19
5.3 Flexbox o Grid . . . 20

6 Herramientas y test de verificación **21**
6.1 El validador de CSS del W3C . . . 21
6.2 Herramientas de desarrollo del navegador . . . 21
6.3 Compatibilidad entre navegadores . . . 22
6.4 Lista de comprobación de una hoja de estilos . . . 22

7 Tecnologías y frameworks **23**
7.1 Diseño adaptable (*responsive*) . . . 23
7.2 Qué es un framework CSS . . . 24
7.3 Bootstrap . . . 24
7.4 Tailwind CSS . . . 25
7.5 Ventajas e inconvenientes de los frameworks . . . 26

8 Preprocesadores de estilos. Variables, mixins y funciones **27**
8.1 Qué es un preprocesador . . . 27
8.2 Variables . . . 27
8.3 Anidación . . . 28
8.4 Mixins . . . 28
8.5 Funciones . . . 29
8.6 Parciales y módulos . . . 30
8.7 Otras características . . . 30
8.8 Compilación . . . 30

Referencias **31**

Esta unidad desarrolla el **Bloque 2** de la programación del módulo y el resultado de aprendizaje **RA2**: *crea interfaces web homogéneos definiendo y aplicando estilos.*

Contenido de la programación	**Criterios de evaluación**
Estilos en línea basados en etiquetas y en clases	RA2.a, RA2.b, RA2.g

Contenido de la programación	**Criterios de evaluación**
Crear y vincular hojas de estilo	RA2.b, RA2.e, RA2.f
Crear y vincular hojas de estilo en cascada externa	RA2.c, RA2.d, RA2.e
Propiedades de los elementos y maquetación (se aplican con cualquiera de las tres formas anteriores)	RA2.f, RA2.g
Herramientas y test de verificación	RA2.h
Tecnologías y frameworks	RA2.i
Preprocesadores de estilos. Variables, mixins y funciones	RA2.j

Las secciones 4 y 5 no son contenidos independientes de la programación: recogen las **propiedades** que se aplican con las hojas de estilo de las secciones 1 a 3 (criterio RA2.f). La referencia general de toda la unidad es la documentación de MDN [1] y las especificaciones del W3C que se citan en cada apartado.

1 Estilos en línea basados en etiquetas y en clases

1.1 Separar contenido y presentación

Una página web se construye con dos lenguajes que tienen funciones distintas:

- **HTML** define la **estructura y el significado** del contenido: qué es un título, un párrafo, una lista, un enlace.
- **CSS** (*Cascading Style Sheets*, hojas de estilo en cascada) define la **presentación**: colores, tipografía, espacios y distribución en la pantalla.

En las primeras versiones de HTML la presentación se escribía en el propio HTML, con etiquetas como `<font>` o `<center>` y atributos como `bgcolor` o `align`. Esas etiquetas y atributos están **obsoletos** en HTML5 [2]: el navegador todavía los entiende, pero no deben usarse.

```
<!-- MAL: presentación dentro del HTML (obsoleto) -->
<center><font color="red" size="5">Oferta de otoño</font></center>

<!-- BIEN: el HTML dice qué es; el CSS dice cómo se ve -->
<h2 class="oferta">Oferta de otoño</h2>
```

```
.oferta { color: #b42318; font-size: 1.5rem; text-align: center; }
```

Separar contenido y presentación permite cambiar el aspecto de todo un sitio editando un solo archivo, reutilizar los estilos, reducir el tamaño del HTML y ofrecer presentaciones distintas del mismo contenido (pantalla, impresión, modo oscuro).

1.2 Las etiquetas HTML y su presentación por defecto

Aunque una página no tenga CSS, el navegador aplica su propia **hoja de estilos del navegador** (*user agent stylesheet*). Por eso, sin escribir nada, un `<h1>` aparece grande y en negrita, un enlace azul y subrayado, y el `<body>` tiene un margen de 8 píxeles.

Cada etiqueta tiene además un **tipo de caja** por defecto:

Tipo	Comportamiento	Etiquetas
Bloque (*block*)	Empieza en una línea nueva y ocupa todo el ancho disponible	`h1`–`h6`, `p`, `div`, `ul`, `li`, `header`, `main`, `section`, `form`
En línea (*inline*)	Se coloca dentro de la línea de texto y ocupa solo lo que mide su contenido	`a`, `span`, `strong`, `em`, `code`, `img`, `label`

1.3 Posibilidades de modificar las etiquetas HTML

Con CSS se puede modificar **todo el aspecto** de cualquier etiqueta (criterio RA2.a):

- su **tipografía**: familia, tamaño, peso, interlineado, alineación;
- sus **colores** y su **fondo**;
- sus **dimensiones** y **espacios** (modelo de caja);
- su **tipo de caja**: con `display: flex` una lista `<ul>` se convierte en un menú horizontal, y con `display: inline-block` un enlace puede mostrarse como un botón;
- su **posición** en la página, e incluso **ocultarla**;
- su aspecto según el **estado** (al pasar el puntero, al recibir el foco) o el **dispositivo** (móvil, impresora).

Lo que **no** debe hacerse es elegir una etiqueta por su aspecto. La etiqueta se elige por su **significado**, y el aspecto se cambia con CSS.

```
<!-- MAL: se usa h1 solo porque "se ve grande" y br para separar -->
<h1>Envío gratuito a partir de 30 euros</h1><br><br>

<!-- BIEN: es un párrafo destacado; el tamaño y el espacio los pone el CSS -->
<p class="destacado">Envío gratuito a partir de 30 euros</p>
```

Los lectores de pantalla y los buscadores interpretan las etiquetas por su significado: un `<h1>` usado como adorno rompe el índice de títulos de la página [3].

1.4 Atributos globales relacionados con los estilos

Todos los elementos HTML admiten estos atributos [2]:

Atributo	Uso	Ejemplo
`class`	Asigna una o varias clases, separadas por espacios	`class="aviso"`
`id`	Identificador **único** en la página	`id="contacto"`
`style`	Estilos en línea para ese elemento	`style="color: red"`
`hidden`	Oculta el elemento	`hidden`
`lang`	Idioma del contenido	`lang="en"`

1.5 Sintaxis de una regla CSS

Una hoja de estilos es una lista de **reglas**. Cada regla tiene un **selector**, que indica a qué elementos se aplica, y un **bloque de declaraciones** entre llaves. Cada declaración es una pareja **propiedad: valor** terminada en punto y coma.

```
/* comentario: el navegador lo ignora */
h2 {                        /* selector */
```

```
  color: #1d4ed8;          /* declaración = propiedad: valor; */
  font-size: 1.5rem;
  margin-bottom: 8px;
}
```

- Si una propiedad o un valor no existen, el navegador **ignora esa declaración** y continúa con la siguiente. No se produce ningún error visible: por eso hay que validar el código.
- Los espacios y saltos de línea no importan, pero el código se escribe con sangría para que se lea bien.
- Los nombres de las propiedades se escriben en minúsculas y con guiones, como `font-size`.

1.6 Estilos en línea: el atributo `style`

La forma más directa de dar estilo a un elemento es escribir las declaraciones en su atributo `style` (criterio RA2.b):

```
<p style="color: #b42318; font-weight: 700;">Plazas agotadas</p>
```

Ventajas	Inconvenientes
Rápido para una prueba	Solo afecta a ese elemento: no se reutiliza
No necesita selector	Mezcla contenido y presentación
Tiene prioridad sobre casi todas las demás reglas	Por esa prioridad, es difícil de sobrescribir después
	No admite pseudoclases (`:hover`) ni consultas de medios

Usos justificados del atributo `style`:

- correos electrónicos en HTML, porque muchos clientes de correo ignoran las hojas de estilo;
- valores que calcula JavaScript, como la posición de un elemento que se arrastra;
- dar valor a una variable CSS en un elemento concreto, como `style="--progreso: 70%"` en una barra de progreso.

1.7 Estilos basados en etiquetas: el selector de tipo

Un **selector de tipo** (o de etiqueta) es el nombre de una etiqueta HTML. La regla se aplica a **todos** los elementos de ese tipo de la página:

```
p  { line-height: 1.5; }          /* todos los párrafos */
a  { color: #1d4ed8; }            /* todos los enlaces */
h1, h2, h3 { font-weight: 700; }  /* agrupación: selectores separados por comas */
*  { box-sizing: border-box; }    /* selector universal: todos los elementos */
```

Los selectores de tipo sirven para fijar el **estilo base** del sitio: la tipografía general, los títulos y los enlaces. Al hacerlo se **redefine** el estilo que el navegador da por defecto a esas etiquetas.

1.8 Estilos basados en clases

Una **clase** es un nombre que se asigna a uno o varios elementos con el atributo `class`. En CSS se selecciona con un **punto** delante (criterio RA2.g):

```
<p class="aviso">Biblioteca cerrada el 12 de octubre.</p>
<p>Horario habitual de 9:00 a 21:00.</p>
<p class="aviso aviso--urgente">Cambio de horario a partir del lunes.</p>
```

```
.aviso {                          /* todos los elementos con la clase aviso */
  padding: 12px 16px;
  border-left: 4px solid #f59e0b;
  background: #fff7e6;
}
.aviso--urgente {                 /* solo los que además tienen esta clase */
  border-left-color: #b42318;
}
p.aviso { font-weight: 700; }     /* solo los p que tienen la clase aviso */
```

- Un elemento puede tener **varias clases**, separadas por espacios, y recibe los estilos de todas.
- La misma clase puede usarse en **elementos distintos**: un `p` y un `div` pueden tener la clase `aviso`.
- `p.aviso` (sin espacio) significa «párrafo que tiene la clase aviso»; `p .aviso` (con espacio) significa «elemento con clase aviso que está **dentro** de un párrafo».

1.9 El selector de identificador

El **identificador** (`id`) es único en cada página. En CSS se selecciona con una **almohadilla**:

```
#contacto { scroll-margin-top: 80px; }
```

En la práctica se prefieren las clases para dar estilo, porque se reutilizan y tienen menos prioridad, lo que facilita sobrescribirlas. El `id` se reserva para los **anclas** de enlaces internos (`<a href="#contacto">`), para unir un `<label for="…">` con su campo y para JavaScript.

1.10 Nombrar clases: buenas prácticas y BEM

Una clase se nombra por **lo que es o la función que cumple**, no por su aspecto. Si mañana el aviso pasa a ser azul, una clase llamada `.rojo` deja de tener sentido.

Mal	Bien	Por qué
`.rojo`, `.letra-grande`	`.aviso-error`, `.titular`	Describe la función, no el aspecto
`.caja1`, `.caja2`	`.tarjeta`, `.tarjeta-destacada`	Se entiende sin mirar el HTML
`.MenuPrincipal`, `.menu_principal`	`.menu-principal`	Convención habitual: minúsculas y guiones
`.izquierda`	`.barra-lateral`	En móvil puede dejar de estar a la izquierda

En proyectos grandes se usan **metodologías de nombrado**. La más extendida es **BEM** (*Block, Element, Modifier*) [4]:

- **Bloque:** un componente independiente: `.tarjeta`.
- **Elemento:** una parte del bloque, unida con dos guiones bajos, como `.tarjeta__titulo`.
- **Modificador:** una variante, unida con dos guiones, como `.tarjeta--destacada`.

```
<article class="tarjeta tarjeta--destacada">
  <h3 class="tarjeta__titulo">Taller de cómic</h3>
  <p class="tarjeta__texto">Cuatro sesiones de guion y dibujo.</p>
  <a class="tarjeta__boton" href="#">Inscribirse</a>
</article>
```

Con BEM cada pieza tiene una clase propia, los selectores son cortos y de baja prioridad, y el nombre indica a qué componente pertenece cada regla.

Ideas clave. HTML define el significado y CSS la presentación; las etiquetas de presentación de HTML están obsoletas. Una regla CSS tiene selector y declaraciones `propiedad: valor;`. El atributo `style` aplica estilos en línea a un solo elemento. El selector de tipo afecta a todas las etiquetas iguales y redefine el estilo del navegador. Las clases (`.nombre`) se reutilizan y se nombran por su función; BEM: `bloque__elemento--modificador`.

2 Crear y vincular hojas de estilo

2.1 Hoja de estilos interna: el elemento `style`

Una **hoja interna** se escribe dentro de un elemento `<style>`, normalmente en el `<head>` del documento:

```
<head>
  <meta charset="utf-8">
  <title>Horario</title>
  <style>
    body { font-family: Arial, sans-serif; }
    .aviso { color: #b42318; }
  </style>
</head>
```

Se aplica a **toda la página** donde está escrita, pero **no se comparte** con otras páginas. Es útil para pruebas, para páginas únicas (una página de error, una página de aterrizaje) y para el CSS mínimo que debe cargarse antes que nada.

2.2 Selectores combinados

Los **combinadores** seleccionan elementos según su posición respecto a otros [5]:

Combinador	Ejemplo	Selecciona
Descendiente (espacio)	`nav a`	Los enlaces que están en cualquier nivel dentro de un `nav`
Hijo (`>`)	`ul > li`	Los `li` que son hijos directos de un `ul`
Hermano adyacente (`+`)	`h2 + p`	El párrafo que va justo después de un `h2`
Hermanos generales (`~`)	`h2 ~ p`	Todos los párrafos que van después de un `h2`, con el mismo padre

```
.menu a { text-decoration: none; }        /* solo los enlaces del menú */
.articulo h2 + p { font-size: 1.125rem; } /* la entradilla tras cada título */
```

2.3 Selectores de atributo

Seleccionan elementos según sus **atributos** o el valor de estos:

Selector	Selecciona
`[title]`	Elementos que tienen el atributo `title`
`[type="email"]`	Campos cuyo `type` es exactamente `email`
`[href^="https"]`	Enlaces cuyo `href` **empieza** por `https`
`[href$=".pdf"]`	Enlaces cuyo `href` **termina** en `.pdf`
`[href*="ejemplo"]`	Enlaces cuyo `href` **contiene** `ejemplo`

```
a[href$=".pdf"]::after { content: " (PDF)"; }   /* avisa de que descarga un PDF */
input[type="email"] { width: 100%; }
```

2.4 Pseudoclases

Una **pseudoclase** selecciona elementos según su **estado** o su **posición**. Se escribe con dos puntos:

Grupo	Pseudoclases	Ejemplo de uso
Enlaces	`:link`, `:visited`	Enlaces no visitados y visitados
Interacción	`:hover`, `:focus`, `:focus-visible`, `:active`	Puntero encima, foco, foco con teclado, pulsado
Estructura	`:first-child`, `:last-child`, `:nth-child(2n)`, `:only-child`	Primera fila, filas pares de una tabla
Formularios	`:checked`, `:disabled`, `:required`, `:invalid`	Casilla marcada, campo obligatorio
Lógicas	`:not()`, `:is()`, `:where()`, `:has()`	Negación, agrupación, «contiene a»

```
.boton:hover { background: #1e40af; }                 /* puntero encima */
.boton:focus-visible { outline: 3px solid #f59e0b; } /* foco con teclado */
tr:nth-child(even) { background: #f3f4f6; }           /* filas pares */
li:not(:last-child) { margin-bottom: 8px; }           /* todos menos el último */
.tarjeta:has(img) { padding-top: 0; }                 /* tarjetas que tienen imagen */
```

En los enlaces, las pseudoclases se escriben en el orden `:link`, `:visited`, `:hover`, `:active` para que ninguna anule a la siguiente.

Foco visible. El navegador dibuja un contorno alrededor del elemento que tiene el foco del teclado. Eliminarlo impide saber dónde se está al navegar con la tecla Tab, lo que incumple el criterio 2.4.7 *Foco visible* de las WCAG 2.2 [3]:

```
/* MAL: el usuario de teclado no sabe dónde está */
a:focus { outline: none; }

/* BIEN: se cambia el aspecto del foco, pero se mantiene visible */
a:focus-visible { outline: 3px solid #f59e0b; outline-offset: 2px; }
```

2.5 Pseudoelementos

Un **pseudoelemento** selecciona una **parte** de un elemento, o crea un contenido que no está en el HTML. Se escribe con **dos** pares de dos puntos:

Pseudoelemento	Selecciona
`::before`, `::after`	Un contenido generado antes o después del contenido del elemento (necesita `content`)
`::first-letter`	La primera letra (letra capital)
`::first-line`	La primera línea del párrafo
`::marker`	La viñeta o el número de un elemento de lista
`::placeholder`	El texto de ejemplo de un campo de formulario
`::selection`	El texto que el usuario selecciona

```
.cita::before { content: "«"; }
.cita::after  { content: "»"; }
.articulo p:first-of-type::first-letter { font-size: 3rem; float: left; }
li::marker { color: #1d4ed8; }
```

El contenido generado con `::before` y `::after` no siempre lo leen los lectores de pantalla: debe ser **decorativo**. La información importante va en el HTML.

2.6 La cascada

Es habitual que varias reglas intenten dar un valor distinto a la misma propiedad del mismo elemento. La **cascada** es el algoritmo que decide cuál gana [6]. Se comprueban, por este orden:

1. **Origen e importancia.** Las declaraciones del autor (la hoja de la web) ganan a las del navegador. Una declaración marcada con `!important` gana a las normales.
2. **Especificidad.** Gana el selector más específico.
3. **Orden de aparición.** Si empatan, gana la regla escrita **más tarde** (más abajo en la hoja, o en una hoja que se carga después).

```
p { color: #1f2430; }
p { color: #4b5563; }      /* misma especificidad: gana esta, que va después */
```

Las especificaciones recientes añaden las **capas de cascada** (`@layer`), que permiten agrupar reglas y fijar qué grupo tiene prioridad independientemente de la especificidad; las usan algunos frameworks.

2.7 Especificidad

La **especificidad** es el peso de un selector. Se calcula contando tres tipos de componentes y se compara columna a columna, de izquierda a derecha [5]:

- **A:** número de identificadores (`#id`).
- **B:** número de clases, atributos y pseudoclases (`.clase`, `[type]`, `:hover`).
- **C:** número de tipos y pseudoelementos (`p`, `::before`).

Selector	A (id)	B (clase, atributo, pseudoclase)	C (tipo, pseudoelemento)
`p`	0	0	1
`nav a`	0	0	2
`.aviso`	0	1	0
`p.aviso`	0	1	1
`.menu a:hover`	0	2	1
`input[type="email"]`	0	1	1
`#contacto p`	1	0	1
`#contacto .aviso`	1	1	0

- Una sola clase (0,1,0) gana a cualquier número de etiquetas (0,0,10), porque la columna B se compara antes que la C.
- Un `id` gana a cualquier número de clases.
- El atributo `style` gana a todos los selectores, y `!important` gana incluso al atributo `style`.
- El selector universal `*` y los combinadores no suman.
- `:where()` siempre suma cero; `:is()` y `:not()` suman lo que su argumento más específico.

```
#principal p { color: #1f2430; }   /* (1,0,1) */
.texto { color: #b42318; }         /* (0,1,0): pierde aunque vaya después */
```

`!important`. Rompe el orden normal de la cascada y obliga a usar otro `!important` para sobrescribirlo. Debe evitarse; solo se justifica en clases de utilidad que siempre deben aplicarse o para sobrescribir estilos de terceros.

2.8 Herencia

Algunas propiedades se **heredan**: si se aplican a un elemento, sus descendientes reciben el mismo valor salvo que otra regla les dé uno propio. Por eso basta con fijar la fuente en el `body` para que la usen todos los párrafos.

Se heredan	No se heredan
`color`, `font-family`, `font-size`, `font-weight`, `line-height`, `text-align`, `letter-spacing`, `list-style`, `visibility`	`margin`, `padding`, `border`, `background`, `width`, `height`, `display`, `position`

Palabras clave que sirven para cualquier propiedad:

- `inherit`: toma el valor del padre;
- `initial`: toma el valor inicial que define la especificación;
- `unset`: hereda si la propiedad es heredable y, si no, toma el valor inicial.

Los campos de formulario (`input`, `button`, `select`) **no heredan** la fuente de la página en la hoja del navegador. Es frecuente corregirlo así:

```
input, button, select, textarea { font: inherit; }
```

2.9 Redefinir estilos

Redefinir un estilo es sustituir el aspecto que ya tiene un elemento, venga de la hoja del navegador, de una hoja anterior o de un framework (criterio RA2.e). Hay tres situaciones habituales.

1. Redefinir el estilo de las etiquetas del navegador. Cada navegador aplica márgenes y tamaños ligeramente distintos. Se igualan con una hoja inicial:

- un **reset** elimina casi todos los estilos del navegador (márgenes, tamaños de títulos, viñetas);
- **Normalize.css** no los elimina, sino que los **iguala** entre navegadores y corrige errores conocidos [7].

```
/* Reset moderno mínimo */
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; }
img { max-width: 100%; height: auto; display: block; }
input, button, select, textarea { font: inherit; }
```

2. Redefinir una regla propia. Para cambiar el aspecto de un componente en un contexto concreto se escribe una regla **más específica** o una regla con la **misma especificidad más abajo**:

```
.boton { background: #1d4ed8; color: #ffffff; }

/* El botón del pie se redefine: misma clase, más específico por el contexto */
.pie .boton {
  background: transparent;
  color: #1d4ed8;
  border: 2px solid currentColor;
}
```

```
/* MAL: escalada de prioridad, cada cambio obliga a subir otra vez */
#pie .boton { background: transparent !important; }

/* BIEN: un modificador explícito, con la misma especificidad que la base */
.boton--secundario { background: transparent; color: #1d4ed8; }
```

3. Redefinir los estilos de un framework. Se carga la hoja propia **después** de la del framework y se usan selectores de la misma especificidad, o se cambian las variables que el propio framework ofrece (sección 7).

Ideas clave. La hoja interna (`<style>`) afecta a una sola página. Combinadores: descendiente, hijo `>`, `+` y `~`. Selectores de atributo, pseudoclases (`:hover`, `:focus-visible`, `:nth-child`) y pseudoelementos (`::before`). La cascada decide por origen e importancia, especificidad (id, clase, tipo) y orden. Algunas propiedades se heredan. Se redefine con reglas más específicas o posteriores, nunca con `!important` como norma; nunca se elimina el foco visible.

3 Crear y vincular hojas de estilo en cascada externa

3.1 La hoja externa y el elemento `link`

Una **hoja externa** es un archivo de texto con extensión `.css` que contiene solo reglas CSS, sin etiquetas HTML. Cada página la **vincula** en su `<head>` con el elemento `<link>` (criterio RA2.c):

```
<head>
  <meta charset="utf-8">
  <title>Biblioteca del barrio</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
```

- `rel="stylesheet"` indica que el archivo enlazado es una hoja de estilos.
- `href` es la ruta del archivo, normalmente **relativa**: `css/estilos.css` desde la raíz del sitio, o `../css/estilos.css` desde una página que está en una subcarpeta.

Son **estilos globales**: una sola hoja da el mismo aspecto a todas las páginas del sitio. Ventajas:

- **Coherencia:** todas las páginas comparten las mismas reglas.
- **Mantenimiento:** un cambio en el archivo se aplica a todo el sitio.
- **Rendimiento:** el navegador guarda la hoja en su **caché** y no la vuelve a descargar en cada página.
- **Trabajo en equipo:** el HTML y el CSS se editan por separado.

Una página puede vincular **varias hojas**. Se aplican en el orden en que aparecen, de modo que, a igual especificidad, la última gana. Por eso la hoja de un framework se coloca antes que la propia:

```
<link rel="stylesheet" href="css/normalize.css">
<link rel="stylesheet" href="css/estilos.css">
```

También puede importarse una hoja desde otra con `@import`, que debe escribirse al principio del archivo. Se usa poco, porque obliga a descargar las hojas una detrás de otra:

```
@import url("base.css");
```

3.2 Comparación de las tres formas de aplicar CSS

Forma	Dónde se escribe	Alcance	Prioridad	Uso recomendado
En línea	Atributo `style` del elemento	Un elemento	La más alta	Correos HTML, valores calculados por JavaScript
Interna	Elemento `<style>` en el `<head>`	Una página	Normal	Pruebas, páginas únicas
Externa	Archivo `.css` vinculado con `<link>`	Todo el sitio	Normal	Siempre que sea posible

La hoja interna y la externa tienen la **misma prioridad**: entre ellas decide la especificidad y, si empatan, el orden en que aparecen en el documento.

3.3 Organización de una hoja global

Una hoja externa de un sitio real puede tener cientos de reglas. Para mantenerla ordenada se divide en secciones, de lo general a lo particular:

```
/* 1. VARIABLES: colores, fuentes, espacios */
:root { --color-marca: #1d4ed8; --fuente: "Source Sans 3", Arial, sans-serif; }
```

```css
/* 2. BASE: reset y estilo de las etiquetas */
body { margin: 0; font-family: var(--fuente); line-height: 1.5; }

/* 3. ESTRUCTURA: cabecera, contenido, pie */
.cabecera { display: flex; justify-content: space-between; }

/* 4. COMPONENTES: botones, tarjetas, avisos */
.boton { padding: 10px 20px; }

/* 5. UTILIDADES: clases pequeñas de un solo uso */
.texto-centrado { text-align: center; }
```

En sitios grandes cada sección puede ser un archivo (`base.css`, `componentes.css`), que se unen al publicar la web, por ejemplo con un preprocesador (sección 8).

3.4 Hojas de estilo alternativas

Una **hoja alternativa** ofrece otra presentación del mismo contenido (criterio RA2.d). Hay tres mecanismos.

1. Hojas para cada medio: el atributo `media`. El elemento `<link>` admite un atributo `media` con una consulta de medios. La hoja solo se aplica cuando la consulta se cumple [8]:

```html
<link rel="stylesheet" href="css/estilos.css">
<link rel="stylesheet" href="css/impresion.css" media="print">
<link rel="stylesheet" href="css/ancho.css" media="(min-width: 64em)">
```

Los tipos de medio vigentes son `screen` (pantallas), `print` (impresión y vista previa de impresión) y `all`. El mismo efecto se consigue dentro de una hoja con la regla `@media`:

```css
@media print {
  .cabecera, nav, .boton { display: none; }   /* no se imprimen */
  body { color: #000000; background: #ffffff; font-size: 12pt; }
  a[href^="http"]::after { content: " (" attr(href) ")"; } /* escribe la URL */
}
```

2. Hojas según las preferencias del usuario. El sistema operativo comunica al navegador algunas preferencias, que CSS puede consultar [9]:

Preferencia del usuario	Consulta
Tema oscuro	`(prefers-color-scheme: dark)`
Menos animaciones	`(prefers-reduced-motion: reduce)`
Más contraste	`(prefers-contrast: more)`

```css
:root { --fondo: #ffffff; --texto: #1f2430; }
@media (prefers-color-scheme: dark) {
  :root { --fondo: #111827; --texto: #e5e7eb; }   /* solo cambian las variables */
}
body { background: var(--fondo); color: var(--texto); }
```

3. Hojas que elige el usuario: `rel="alternate stylesheet"`. HTML permite declarar varias hojas con **nombre** (atributo `title`) para que el usuario elija una [2]:

```html
<!-- persistente: sin title, se aplica siempre -->
<link rel="stylesheet" href="css/base.css">
```

```
<!-- preferida: con title, se aplica por defecto -->
<link rel="stylesheet" href="css/claro.css" title="Claro">
<!-- alternativas: con title, desactivadas hasta que el usuario las elige -->
<link rel="alternate stylesheet" href="css/oscuro.css" title="Oscuro">
<link rel="alternate stylesheet" href="css/contraste.css" title="Alto contraste">
```

- Firefox muestra estas hojas en el menú **Ver › Estilo de página**.
- Chrome y Safari no tienen ese menú. En ellos, el cambio se programa con un pequeño **selector de estilo** en JavaScript, que activa o desactiva cada `<link>` mediante su propiedad `disabled` y guarda la elección del usuario.

Ofrecer una versión de alto contraste o de letra grande ayuda a personas con baja visión, aunque no sustituye a que la hoja principal cumpla por sí misma las WCAG 2.2 [3].

3.5 Propiedades personalizadas (variables CSS)

Las **propiedades personalizadas** (*custom properties*), llamadas habitualmente **variables CSS**, guardan un valor con un nombre que empieza por dos guiones. Se leen con la función `var()` [10]:

```
:root {                      /* :root es el elemento html: variables globales */
  --color-marca: #1d4ed8;
  --espacio: 16px;
}
.boton {
  background: var(--color-marca);
  padding: calc(var(--espacio) / 2) var(--espacio);
  color: var(--color-texto-boton, #ffffff);   /* segundo valor: el de reserva */
}
```

- Se **heredan** y siguen la cascada: se pueden redefinir en un componente o en una consulta de medios.
- Su valor puede cambiarse en el navegador mientras la página está abierta (tema oscuro, JavaScript).
- Son la forma de llevar al código los colores, fuentes y espacios de la **guía de estilo** de la UT1.

```
.tarjeta--oferta { --color-marca: #b42318; }  /* aquí, la marca es roja */
```

Ideas clave. La hoja externa (`<link rel="stylesheet" href="…">`) da estilos globales a todo el sitio, se guarda en caché y facilita el mantenimiento. Varias hojas se aplican en orden. Hojas alternativas: por medio (`media="print"`), por preferencia (`prefers-color-scheme`) y elegibles por el usuario (`rel="alternate stylesheet"` con `title`). Las variables CSS (`--nombre`, `var()`) se heredan y pueden cambiar con la cascada.

4 Propiedades de los elementos

Cada elemento tiene decenas de propiedades que se pueden modificar (criterio RA2.f). En esta sección se agrupan las más utilizadas. La referencia completa de cada una, con sus valores y su compatibilidad, está en MDN [1] y en el manual de Meyer y Weyl [11].

4.1 Valores y unidades

Unidad	Relativa a	Uso habitual
`px`	Un píxel de referencia (unidad absoluta)	Bordes, sombras, detalles finos
`rem`	Tamaño de letra del elemento raíz (16 px por defecto)	Tamaños de letra, espacios generales
`em`	Tamaño de letra del propio elemento	Relleno de botones que crece con su texto
`%`	Medida del elemento padre	Anchos fluidos
`vw`, `vh`	1 % del ancho o del alto de la ventana	Secciones a pantalla completa
`ch`	Ancho del carácter «0» de la fuente	Longitud de línea: `max-width: 65ch`
`fr`	Fracción del espacio libre (solo en Grid)	Columnas de una rejilla

- Los tamaños de letra se expresan en `rem` para respetar el tamaño que el usuario configura en su navegador.
- `calc()` combina unidades: `width: calc(100% - 2rem)`.
- `clamp(1.5rem, 4vw, 2.5rem)` da un tamaño que crece con la ventana, pero nunca baja de 1,5 rem ni pasa de 2,5 rem: `clamp()` limita un valor fluido entre un mínimo y un máximo.
- Un valor `0` no necesita unidad. Cualquier otro número, sí: `margin: 10` es un **error** y se ignora.

4.2 Color y fondos

Propiedad	Función	Valor de ejemplo
`color`	Color del texto (se hereda)	`#1f2430`
`background-color`	Color de fondo	`hsl(225 80% 96%)`
`background-image`	Imagen o degradado de fondo	`url("img/fondo.jpg")`
`background-size`	Tamaño de la imagen de fondo	`cover` (cubre la caja), `contain` (cabe entera)
`background-position`	Posición de la imagen	`center`, `top right`
`background-repeat`	Repetición	`no-repeat`
`opacity`	Transparencia del elemento entero (0 a 1)	`0.6`

- Formatos de color: nombre (`white`), hexadecimal (`#ffffff`), `rgb()` y `hsl()`, todos con un cuarto valor opcional de transparencia: `rgb(0 0 0 / 0.5)`.
- `currentColor` toma el valor de `color` del propio elemento: útil para bordes e iconos.
- `linear-gradient(#1d4ed8, #7c3aed)` es un **degradado**: una imagen generada por CSS.
- La propiedad abreviada `background` admite todos los valores a la vez.

Texto sobre una imagen. El contraste debe cumplirse en cualquier punto de la imagen (criterio 1.4.3 de las WCAG 2.2 [3]). La solución habitual es oscurecer la imagen con un degradado:

```
.portada {
  background: linear-gradient(rgb(0 0 0 / 0.55), rgb(0 0 0 / 0.55)),
              url("img/portada.jpg") center / cover no-repeat;
```

```
  color: #ffffff;
}
```

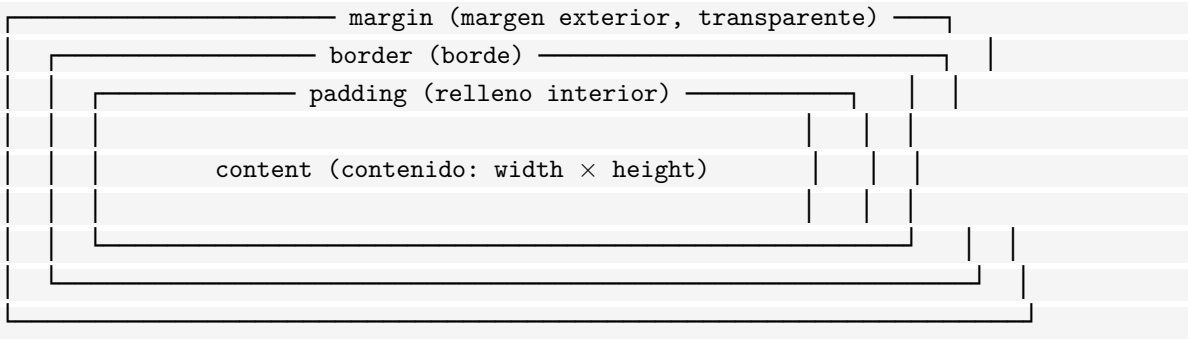
4.3 Texto y fuentes

Propiedad	Función	Valores habituales
`font-family`	Familia, con una pila de respaldo	`Arial, sans-serif`
`font-size`	Tamaño	`1rem`, `1.25rem`
`font-weight`	Peso (grosor)	`400` normal, `700` negrita (de 100 a 900)
`font-style`	Estilo	`normal`, `italic`
`line-height`	Interlineado	`1.5`, sin unidad: proporcional al tamaño
`text-align`	Alineación	`left`, `center`, `right`, `justify`
`text-decoration`	Subrayado y tachado	`none`, `underline`
`text-transform`	Mayúsculas	`uppercase`, `capitalize`
`letter-spacing`	Espacio entre letras	`0.05em`

- La pila de `font-family` termina siempre en una **familia genérica** (`serif`, `sans-serif`, `monospace`), que el navegador sustituye por una fuente instalada.
- Las **fuentes web** se cargan con la regla `@font-face` o con el `<link>` que proporciona un servicio como Google Fonts [12]. Hay que cargar solo los pesos que se usen.
- La propiedad abreviada `font` reúne varias: `font: 700 1.25rem/1.2 var(--fuente);` (peso, tamaño/interlineado y familia).

4.4 El modelo de caja

Todo elemento se dibuja como una **caja rectangular** formada por cuatro zonas, de dentro afuera [13]:



Zona	Propiedades	Observaciones
Contenido	`width`, `height`, `min-width`, `max-width`	`max-width` evita que el contenido se estire demasiado
Relleno	`padding`	Toma el color de fondo del elemento

Zona	Propiedades	Observaciones
Borde	`border`, `border-radius`	`border: 2px solid #1d4ed8;` (grosor, estilo, color)
Margen	`margin`	Siempre transparente; `margin: 0 auto` centra un bloque con ancho fijo

Valores abreviados. `margin` y `padding` admiten de uno a cuatro valores, que se leen en el sentido de las agujas del reloj empezando por arriba:

```
padding: 16px;               /* los cuatro lados */
padding: 8px 16px;           /* arriba y abajo 8 · derecha e izquierda 16 */
padding: 8px 16px 24px;      /* arriba 8 · lados 16 · abajo 24 */
padding: 8px 16px 24px 32px; /* arriba · derecha · abajo · izquierda */
```

`box-sizing`. Por defecto (`content-box`), `width` mide solo el contenido, y el relleno y el borde se **suman**. Con `border-box`, `width` incluye el relleno y el borde, lo que facilita los cálculos:

```
.caja { width: 300px; padding: 20px; border: 2px solid; }
```

`box-sizing`	Ancho total de la caja	Ancho del contenido
`content-box` (por defecto)	300 + 40 + 4 = **344 px**	300 px
`border-box`	**300 px**	300 − 40 − 4 = 256 px

Por eso casi todas las hojas empiezan con esta regla:

```
*, *::before, *::after { box-sizing: border-box; }
```

Otros aspectos del modelo de caja:

- **Colapso de márgenes:** el margen inferior de un bloque y el superior del siguiente no se suman; se aplica el mayor.
- `box-shadow: 0 4px 12px rgb(0 0 0 / 0.15)` dibuja una sombra (desplazamiento horizontal y vertical, desenfoque y color).
- `outline` dibuja un contorno que **no ocupa espacio**; es el que usa el navegador para el foco.
- `overflow` decide qué ocurre si el contenido no cabe: `visible`, `hidden`, `auto` (barra de desplazamiento si hace falta).

Las herramientas de desarrollo del navegador muestran el modelo de caja de cada elemento con sus medidas (sección 6).

4.5 Tipo de visualización: `display`

Valor	Comportamiento
`block`	Caja de bloque: línea propia, todo el ancho, admite `width` y `height`
`inline`	Caja en línea: no admite `width` ni `height`, ni márgenes verticales
`inline-block`	En línea, pero admite medidas y márgenes: botones, etiquetas
`none`	El elemento no se muestra ni ocupa espacio
`flex`	Sus hijos se distribuyen con Flexbox (sección 5)

Valor	Comportamiento
`grid`	Sus hijos se distribuyen en una rejilla (sección 5)

`visibility: hidden` oculta el elemento **pero conserva su hueco**; `display: none` lo quita por completo. Ambos lo ocultan también a los lectores de pantalla.

4.6 Posicionamiento

La propiedad `position` saca un elemento de su lugar normal. Se combina con `top`, `right`, `bottom`, `left` y `z-index`:

Valor	Se coloca respecto a	Uso típico
`static`	Flujo normal (valor por defecto)	—
`relative`	Su posición normal; conserva su hueco	Servir de referencia a hijos absolutos
`absolute`	El antepasado posicionado más cercano; deja de ocupar hueco	Etiqueta «Nuevo» sobre una tarjeta
`fixed`	La ventana; no se mueve al desplazar	Botón de chat, aviso de cookies
`sticky`	Fluye normal hasta un umbral y luego se queda fijo	Cabecera que se queda arriba

```
.tarjeta { position: relative; }      /* referencia para la etiqueta */
.tarjeta__etiqueta {
  position: absolute;
  top: 12px;
  right: 12px;                        /* esquina superior derecha de la tarjeta */
}
.cabecera { position: sticky; top: 0; z-index: 10; }  /* z-index: por delante */
```

La propiedad `float` se usó durante años para maquetar columnas. Hoy solo se utiliza para lo que fue creada: que el texto **rodee** una imagen.

Ideas clave. Unidades: `rem` para letra, `%`, `vw` y `fr` para anchos, `ch` para líneas; un número distinto de 0 necesita unidad. Modelo de caja: contenido, relleno, borde y margen; con `border-box` el ancho incluye relleno y borde. `display` define el tipo de caja; `position` (`relative`, `absolute`, `fixed`, `sticky`) saca el elemento del flujo.

5 Maquetación con Flexbox y Grid

Maquetar con CSS es distribuir las cajas en la página. Los dos sistemas actuales son Flexbox y Grid. Ambos se activan en un **contenedor** con `display`, y afectan a sus **hijos directos**.

5.1 Flexbox

Flexbox distribuye los elementos en **una dimensión**: en una fila o en una columna [14]. Tiene un **eje principal** (el de `flex-direction`) y un **eje transversal**, perpendicular al anterior.

Propiedad (en el contenedor)	Función	Valores habituales
`display: flex`	Activa Flexbox	—
`flex-direction`	Dirección del eje principal	`row` (fila), `column` (columna)
`flex-wrap`	Permitir varias líneas	`nowrap`, `wrap`
`justify-content`	Reparto en el eje principal	`flex-start`, `center`, `space-between`, `space-around`
`align-items`	Alineación en el eje transversal	`stretch`, `center`, `flex-start`, `flex-end`
`gap`	Espacio entre los elementos	`16px`, `1rem`

Propiedad (en los hijos)	Función
`flex: 1`	El elemento crece para ocupar el espacio libre
`flex: 0 0 200px`	No crece, no encoge y mide 200 px
`align-self`	Alineación propia, distinta de la del resto
`order`	Cambia el orden visual (no el de lectura)

```css
/* Menú horizontal: logotipo a la izquierda, enlaces a la derecha */
.cabecera {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;                /* en pantallas estrechas, el menú baja */
  gap: 16px;
}
/* Tarjeta en columna con el botón siempre abajo */
.tarjeta { display: flex; flex-direction: column; }
.tarjeta__boton { margin-top: auto; }   /* el margen automático lo empuja abajo */
```

`order` y `flex-direction: row-reverse` cambian el orden **visual**, pero el lector de pantalla y la tecla Tab siguen el orden del HTML. Si no coinciden, la navegación resulta confusa (criterio 1.3.2 de las WCAG 2.2) [3].

5.2 Grid

Grid distribuye los elementos en **dos dimensiones**, filas y columnas a la vez [15].

Propiedad	Función	Ejemplo
`display: grid`	Activa la rejilla	—
`grid-template-columns`	Número y ancho de las columnas	`1fr 1fr 1fr`, `repeat(3, 1fr)`, `200px 1fr`
`grid-template-rows`	Alto de las filas	`auto 1fr auto`
`gap`	Espacio entre filas y columnas	`24px`

Propiedad	Función	Ejemplo
`grid-column`	Columnas que ocupa un hijo	`span 2`, `1 / -1` (todo el ancho)
`grid-template-areas`	Rejilla con zonas con nombre	Ver el ejemplo
`place-items`	Alineación de los hijos en su celda	`center`

```css
/* Galería adaptable sin consultas de medios */
.galeria {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 24px;
}
```

`repeat(auto-fit, minmax(220px, 1fr))` significa: «tantas columnas como quepan, de 220 px como mínimo, y repartid el sobrante a partes iguales». En un móvil habrá una columna y en un monitor ancho, cuatro o cinco.

```css
/* Estructura de página con áreas con nombre */
.pagina {
  display: grid;
  grid-template-columns: 1fr 280px;
  grid-template-areas:
    "cabecera  cabecera"
    "principal lateral"
    "pie       pie";
  gap: 24px;
}
.cabecera { grid-area: cabecera; }
.principal { grid-area: principal; }
.lateral { grid-area: lateral; }
.pie { grid-area: pie; }
```

5.3 Flexbox o Grid

Criterio	Flexbox	Grid
Dimensiones	Una: fila o columna	Dos: filas y columnas
Punto de partida	El contenido: cada elemento mide lo que necesita	La estructura: se definen las columnas y se colocan los elementos
Usos típicos	Menús, barras de botones, alinear un icono con su texto, el interior de una tarjeta	Estructura de la página, galerías, rejillas de tarjetas, formularios en columnas

Se combinan con frecuencia: Grid para la estructura de la página y Flexbox dentro de cada componente.

Centrar un elemento en horizontal y vertical, un problema clásico, se resuelve hoy con dos líneas:

```css
.contenedor { display: grid; place-items: center; min-height: 100vh; }
```

Ideas clave. Flexbox trabaja en una dimensión, con eje principal (`justify-content`) y transversal (`align-items`). Grid trabaja en dos: `grid-template-columns` con `fr`, `repeat()` y `minmax()`, y `grid-template-areas`. `gap` separa los elementos en ambos. Cambiar el orden visual no cambia el orden de lectura.

6 Herramientas y test de verificación

Un CSS con errores no produce mensajes en la página: el navegador **ignora** lo que no entiende y sigue adelante. Por eso hay que **verificar** las hojas de estilo con herramientas específicas (criterio RA2.h).

6.1 El validador de CSS del W3C

El **CSS Validation Service** del W3C comprueba que una hoja cumple las especificaciones [16]. Admite tres formas de entrada:

- **por URI:** la dirección de una página o de una hoja publicada;
- **por carga de archivo:** se sube el `.css` desde el ordenador;
- **por entrada directa:** se pega el código en un cuadro de texto.

El resultado distingue:

- **errores:** propiedades inexistentes, valores no válidos o errores de sintaxis; deben corregirse todos;
- **advertencias:** código válido que puede dar problemas, como los prefijos de proveedor, o que el validador no puede comprobar, como los valores de las variables CSS; se revisan, pero no siempre requieren cambios.

Errores frecuentes que detecta el validador, con su corrección:

```
colr: red;           /* propiedad inexistente          → color: red;      */
margin: 10;          /* falta la unidad                → margin: 10px;    */
font-size: 16 px;    /* espacio entre número y unidad  → font-size: 16px; */
color: #12345;       /* hexadecimal de cinco cifras    → color: #123456;  */
display: flexbox;    /* valor no válido                → display: flex;   */
padding: 8px         /* falta el punto y coma: se pierden esta declaración */
color: red;          /* y la siguiente                                    */
.menu { color: red;  /* llave sin cerrar: se pierde el resto de la hoja   */
```

El validador del W3C también tiene limitaciones:

- una hoja **válida** no es necesariamente una hoja **bien diseñada**, ni accesible;
- las propiedades más recientes pueden tardar en incorporarse al validador y aparecer como advertencias o errores, aunque los navegadores ya las admitan; en ese caso se comprueba su estado en MDN y en Can I use.

El HTML se valida con el **Nu Html Checker** del W3C [17]. Validarlo también es importante para el CSS: una etiqueta mal cerrada cambia la estructura del documento y los selectores dejan de seleccionar lo esperado.

6.2 Herramientas de desarrollo del navegador

Las **herramientas de desarrollo** (*DevTools*, tecla F12) permiten inspeccionar y modificar el CSS de cualquier página mientras se ve [18]:

Panel o función	Para qué sirve
Elements › Styles	Muestra las reglas que se aplican al elemento elegido, en orden de prioridad. Las declaraciones anuladas por la cascada aparecen **tachadas**; las no válidas, con un aviso. Permite editar valores y activar o desactivar declaraciones.
Elements › Computed	Valor final de cada propiedad y la regla de la que procede. Incluye el diagrama del modelo de caja con sus medidas.
Insignias `flex` y `grid`	Dibujan sobre la página las líneas de la rejilla y los huecos.
Device Mode (Ctrl + Mayús + M)	Simula el ancho de distintos dispositivos para comprobar el diseño adaptable.
Rendering	Emula el medio `print` y preferencias como `prefers-color-scheme: dark` o `prefers-reduced-motion`.
Lighthouse	Auditoría de rendimiento, accesibilidad y buenas prácticas.

Al pasar el puntero sobre un selector en el panel **Styles**, Chrome muestra su especificidad.

6.3 Compatibilidad entre navegadores

No todos los navegadores admiten todas las propiedades, o no desde la misma versión. Antes de usar una propiedad reciente se comprueba su compatibilidad:

- **Can I use** [19] muestra para cada propiedad una tabla por navegador y versión: en verde, compatible; en rojo, no compatible; en amarillo, compatibilidad parcial. Indica también el porcentaje de usuarios que podrán verla.
- Las páginas de MDN terminan con una tabla de **compatibilidad con navegadores** [1]. MDN y Can I use indican también si una característica forma parte de **Baseline**, el conjunto que ya funciona en todos los navegadores principales.

Si una propiedad no es compatible con todos los navegadores, existen dos recursos:

- **Prefijos de proveedor** (`-webkit-`, `-moz-`): versiones experimentales de una propiedad. Hoy se añaden de forma automática con herramientas como Autoprefixer y casi no hacen falta.
- **Mejora progresiva** con `@supports`: se escribe un estilo básico para todos y se mejora donde se admite.

```
.galeria { display: flex; flex-wrap: wrap; }     /* para todos */
@supports (display: grid) {
  .galeria { display: grid; grid-template-columns: repeat(3, 1fr); }
}
```

Por último, el resultado se prueba en **varios navegadores** (Chrome, Firefox, Edge y, si es posible, Safari) y en dispositivos reales o simulados.

6.4 Lista de comprobación de una hoja de estilos

1. El CSS pasa el validador del W3C sin errores, y el HTML pasa el Nu Html Checker.
2. No quedan declaraciones tachadas por error en DevTools.
3. Los colores de texto cumplen el contraste mínimo (4,5 : 1) [20].
4. El foco del teclado se ve en todos los elementos interactivos.

5. La página se ve bien de 320 px a 1920 px de ancho y al ampliar el texto al 200 %.
6. Las propiedades recientes se han comprobado en Can I use.
7. La página se ha probado en dos navegadores como mínimo.

Ideas clave. El navegador ignora el CSS erróneo sin avisar: hay que validarlo con el CSS Validation Service del W3C (URI, archivo o entrada directa; errores y advertencias). DevTools muestra las reglas aplicadas, las anuladas y el modelo de caja, y simula dispositivos y preferencias. La compatibilidad se consulta en Can I use y MDN, y se resuelve con prefijos o con `@supports`.

7 Tecnologías y frameworks

7.1 Diseño adaptable (*responsive*)

El **diseño adaptable** (*responsive web design*) consigue que una misma página se adapte a cualquier ancho de pantalla. El término lo propuso Ethan Marcotte en 2010 y se basa en tres técnicas [21]:

1. **rejillas fluidas**, con anchos relativos (`%`, `fr`) en lugar de píxeles fijos;
2. **imágenes flexibles**, que nunca superan el ancho de su contenedor;
3. **consultas de medios**, que cambian el estilo a partir de ciertos anchos.

La etiqueta `meta viewport`. Sin ella, los móviles muestran la página como si midiera unos 980 px y la reducen, de modo que el texto se ve diminuto y las consultas de medios no funcionan:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Imágenes flexibles:

```
img, video { max-width: 100%; height: auto; }
```

Consultas de medios. La regla `@media` aplica un grupo de reglas solo si se cumple una condición [8]:

```
@media (min-width: 48em) { ... }                          /* 768 px o más */
@media (min-width: 48em) and (max-width: 63.99em) { ... } /* de 768 a 1023 px */
@media (width >= 48em) { ... }                            /* sintaxis de rango */
@media (orientation: landscape) { ... }                   /* pantalla apaisada */
```

Los **puntos de ruptura** (*breakpoints*) son los anchos en los que cambia el diseño. Se eligen donde el **contenido** lo necesita, no por modelos de dispositivo concretos. Como orientación:

Rango	Dispositivo típico
Menos de 600 px	Móvil en vertical
De 600 a 1023 px	Móvil en horizontal, tableta
1024 px o más	Portátil y sobremesa

Mobile first. Consiste en escribir primero los estilos para la pantalla **pequeña** y añadir después consultas con `min-width` para las grandes [22]. Obliga a priorizar el contenido y produce un CSS más sencillo, porque el móvil suele tener la maquetación más simple (una columna):

```
/* Base: móvil, una columna */
.tarjetas { display: grid; gap: 16px; }
```

```
/* Tableta: dos columnas */
@media (min-width: 40em) {
  .tarjetas { grid-template-columns: 1fr 1fr; }
}
/* Escritorio: tres columnas */
@media (min-width: 64em) {
  .tarjetas { grid-template-columns: repeat(3, 1fr); }
}
```

La estrategia contraria, **desktop first**, parte del escritorio y usa consultas con `max-width` para reducir.

Unidades relativas. Las consultas se suelen escribir en `em` (1 em = 16 px en las consultas), porque así responden también cuando el usuario aumenta el tamaño de letra. Las WCAG 2.2 exigen que el contenido se pueda ver sin desplazamiento horizontal con un ancho de 320 px (criterio 1.4.10 *Reajuste de los elementos*) [3].

7.2 Qué es un framework CSS

Un **framework CSS** es una biblioteca de estilos ya escritos (rejilla, componentes, utilidades) que se incorpora a un proyecto para no empezar de cero. Hay dos enfoques:

Enfoque	Idea	Ejemplos
Basado en componentes	Clases con significado de componente: `.btn`, `.card`, `.navbar`	Bootstrap, Bulma, Foundation
De utilidades (*utility-first*)	Muchas clases pequeñas, cada una con una sola propiedad, que se combinan en el HTML	Tailwind CSS
Sin clases (*classless*)	Da estilo a las etiquetas HTML directamente	Pico CSS, Water.css

7.3 Bootstrap

Bootstrap es el framework CSS más utilizado. Su versión actual es la 5.3 [23].

Incorporación por CDN. Un CDN (*content delivery network*) es un servidor que distribuye archivos públicos. Bootstrap se incorpora con un `<link>` a su hoja y, si se usan componentes interactivos (menú desplegable, ventanas modales), con un `<script>`:

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
<link rel="stylesheet" href="css/estilos.css">   <!-- la hoja propia, después -->
...
<script
  src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>
```

La rejilla. Se basa en **12 columnas**. Un `.container` centra el contenido, cada `.row` es una fila y cada `.col-*` indica cuántas de las 12 columnas ocupa a partir de un punto de ruptura:

Prefijo	Desde	Dispositivo típico
(ninguno) `.col-`	0 px	Móvil
`.col-sm-`	576 px	Móvil en horizontal
`.col-md-`	768 px	Tableta

Prefijo	Desde	Dispositivo típico
`.col-lg-`	992 px	Portátil
`.col-xl-`	1200 px	Sobremesa
`.col-xxl-`	1400 px	Pantalla grande

```
<div class="container">
  <div class="row g-4">                    <!-- g-4: separación entre columnas -->
    <div class="col-12 col-md-6 col-lg-4">...</div>  <!-- 1, 2 o 3 por fila -->
    <div class="col-12 col-md-6 col-lg-4">...</div>
    <div class="col-12 col-md-6 col-lg-4">...</div>
  </div>
</div>
```

La rejilla es *mobile first*: `col-12 col-md-6 col-lg-4` significa «todo el ancho en móvil, media fila desde 768 px y un tercio desde 992 px».

Componentes y utilidades.

- Componentes: barra de navegación (`.navbar`), tarjeta (`.card`), botón (`.btn`, con variantes como `.btn-primary`), aviso (`.alert`), campo de formulario (`.form-control`) y ventana modal (`.modal`).
- Utilidades: espaciado (`mt-3` margen superior, `p-4` relleno, `mb-0`), texto (`text-center`, `fw-bold`), color (`text-bg-primary`), visualización (`d-flex`, `d-none d-md-block`), Flexbox (`justify-content-between`).

Personalización. Bootstrap 5.3 expone sus valores como **variables CSS** con el prefijo `--bs-`. Se redefinen en la hoja propia, cargada después:

```
:root {
  --bs-body-font-family: "Source Sans 3", Arial, sans-serif;
  --bs-link-color-rgb: 29, 78, 216;
}
.btn-primary {                  /* cada componente tiene sus propias variables */
  --bs-btn-bg: #1d4ed8;
  --bs-btn-border-color: #1d4ed8;
  --bs-btn-hover-bg: #1e40af;
}
```

Para una personalización completa (cambiar los colores del tema, eliminar componentes que no se usan) se compila Bootstrap desde su código fuente en Sass.

7.4 Tailwind CSS

Tailwind CSS sigue el enfoque de **utilidades**: no ofrece componentes, sino cientos de clases de una sola propiedad que se combinan en el HTML [24]:

```
<a href="#"
   class="inline-block rounded-lg bg-blue-700 px-5 py-2 font-bold text-white
          hover:bg-blue-800 md:px-8">
  Inscribirse
</a>
```

- `px-5` es un relleno horizontal, `bg-blue-700` un color de fondo de su paleta, `rounded-lg` un radio de borde.
- Los prefijos `hover:`, `focus:` y `md:` aplican la clase en un estado o a partir de un punto de

ruptura (`sm` 640 px, `md` 768 px, `lg` 1024 px, `xl` 1280 px).
- En un proyecto real, Tailwind se instala con una herramienta de compilación que **genera solo las clases que se usan**, por lo que el CSS final es pequeño.
- Para prototipos existe una versión que se carga con un `<script>` desde un CDN y genera los estilos en el navegador; su documentación indica que no es adecuada para producción.

7.5 Ventajas e inconvenientes de los frameworks

Aspecto	**CSS propio**	**Bootstrap**	**Tailwind CSS**
Rapidez para empezar	Baja	Muy alta	Alta
Aspecto final	Totalmente propio	Reconocible si no se personaliza	Propio, pero con su paleta y escala
Tamaño del CSS	Solo lo necesario	Grande si se carga completo	Pequeño tras compilar
HTML resultante	Limpio, clases semánticas	Clases de componentes y de utilidad	Muchas clases en cada elemento
Curva de aprendizaje	Hay que dominar CSS	Baja: copiar componentes de la documentación	Media: memorizar las clases
Dependencia	Ninguna	Del framework y de sus versiones	Del framework y de su herramienta de compilación

Ventajas generales:

- rapidez de desarrollo y prototipado;
- rejilla y componentes **probados** en muchos navegadores;
- coherencia visual en equipos grandes;
- documentación y comunidad amplias.

Inconvenientes generales:

- se carga código que no se usa;
- las webs se parecen entre sí si no se personalizan;
- redefinir sus estilos puede obligar a luchar con la especificidad;
- no sustituye a saber CSS: para personalizar o corregir errores hay que entender lo que el framework hace;
- los componentes no garantizan la accesibilidad: dependen de cómo se escriba el HTML.

Ideas clave. Diseño adaptable: rejillas fluidas, imágenes flexibles y consultas de medios, más la etiqueta `meta viewport`. *Mobile first*: estilos base para móvil y `min-width` para ampliar. Bootstrap: CDN, rejilla de 12 columnas (`container`, `row`, `col-md-6`), componentes y utilidades, personalización con variables `--bs-`. Tailwind: clases de utilidad combinadas en el HTML. Un framework acelera, pero no sustituye al CSS.

8 Preprocesadores de estilos. Variables, mixins y funciones

8.1 Qué es un preprocesador

Un **preprocesador de CSS** es un lenguaje que amplía CSS con funciones de programación (variables, anidación, funciones, bucles) y que se **compila** (traduce) a CSS estándar, el único que entienden los navegadores (criterio RA2.j).

```
estilos.scss  →  compilador de Sass  →  estilos.css  →  navegador
(lo escribe el      (en el ordenador        (lo enlaza
 desarrollador)      o en el editor)          el HTML)
```

El más extendido es **Sass** [25], que tiene dos sintaxis:

- **SCSS** (archivos `.scss`): la misma sintaxis que CSS, con llaves y punto y coma. Todo CSS válido es SCSS válido. Es la que se usa en este módulo.
- **Sintaxis indentada** (archivos `.sass`): sin llaves ni punto y coma; la estructura se marca con la sangría.

La implementación oficial es **Dart Sass**. Otros preprocesadores son Less y Stylus. Se llama **postprocesador** a la herramienta que transforma un CSS ya escrito, como PostCSS con Autoprefixer, que añade los prefijos de proveedor.

8.2 Variables

Las variables de Sass empiezan por `$` y se sustituyen por su valor al compilar:

```
$color-marca: #1d4ed8;
$fuente: "Source Sans 3", Arial, sans-serif;
$radio: 8px;

.boton {
  background: $color-marca;
  border-radius: $radio;
  font-family: $fuente;
}
```

CSS compilado:

```
.boton {
  background: #1d4ed8;
  border-radius: 8px;
  font-family: "Source Sans 3", Arial, sans-serif;
}
```

Variables de Sass y variables CSS no son lo mismo:

Aspecto	Variable de Sass (`$color`)	Variable CSS (`--color`)
Cuándo se resuelve	Al compilar; en el CSS final ya no existe	En el navegador, mientras se ve la página
Cascada y herencia	No	Sí: puede cambiar en un componente o en una consulta de medios
Uso en funciones de Sass	Sí (aclarar un color, operaciones)	No: Sass no conoce su valor

Aspecto	Variable de Sass (`$color`)	Variable CSS (`--color`)
Uso típico	Valores internos del código, puntos de ruptura	Temas (claro u oscuro), valores que cambian

Es frecuente combinarlas: `:root { --color-marca: #{$color-marca}; }` pasa el valor de Sass a una variable CSS (la sintaxis `#{…}` se llama **interpolación**).

8.3 Anidación

Las reglas se pueden escribir **dentro** de otras, siguiendo la estructura del HTML. El símbolo `&` representa el selector padre:

```
.tarjeta {
  padding: 24px;

  h3 { margin-top: 0; }               /* .tarjeta h3 */

  &:hover { box-shadow: 0 4px 12px rgb(0 0 0 / 0.15); }   /* .tarjeta:hover */

  &__boton { background: $color-marca; }   /* .tarjeta__boton (BEM) */

  &--destacada { border: 2px solid $color-marca; }   /* .tarjeta--destacada */
}
```

CSS compilado:

```
.tarjeta { padding: 24px; }
.tarjeta h3 { margin-top: 0; }
.tarjeta:hover { box-shadow: 0 4px 12px rgb(0 0 0 / 0.15); }
.tarjeta__boton { background: #1d4ed8; }
.tarjeta--destacada { border: 2px solid #1d4ed8; }
```

- No conviene anidar más de **tres niveles**: se generan selectores largos y muy específicos, difíciles de redefinir.
- CSS ya admite la **anidación nativa** en los navegadores actuales [19], pero Sass la generaliza y la combina con el resto de sus funciones (`&__elemento` no existe en CSS nativo).

8.4 Mixins

Un **mixin** es un bloque de declaraciones reutilizable, que puede recibir **parámetros**. Se define con `@mixin` y se inserta con `@include`:

```
@mixin boton($fondo, $texto: #ffffff) {   /* $texto tiene un valor por defecto */
  display: inline-block;
  padding: 10px 20px;
  border-radius: 6px;
  background: $fondo;
  color: $texto;
}

.boton-principal  { @include boton(#1d4ed8); }
.boton-secundario { @include boton(#e5e7eb, #1f2430); }
```

CSS compilado:

```
.boton-principal {
```

```
  display: inline-block; padding: 10px 20px; border-radius: 6px;
  background: #1d4ed8; color: #ffffff;
}
.boton-secundario {
  display: inline-block; padding: 10px 20px; border-radius: 6px;
  background: #e5e7eb; color: #1f2430;
}
```

Un mixin muy útil agrupa los **puntos de ruptura**. La directiva `@content` inserta las reglas que se escriben al usarlo:

```
@mixin desde($ancho) {
  @media (min-width: $ancho) { @content; }
}
.tarjetas {
  display: grid;
  @include desde(48em) { grid-template-columns: 1fr 1fr; }
}
```

CSS compilado:

```
.tarjetas { display: grid; }
@media (min-width: 48em) {
  .tarjetas { grid-template-columns: 1fr 1fr; }
}
```

8.5 Funciones

Una **función** recibe valores y **devuelve un valor**. Sass incluye módulos de funciones, que se cargan con `@use` al principio del archivo:

```
@use "sass:color";
@use "sass:math";

$marca: #1d4ed8;

.boton:hover {
  background: color.adjust($marca, $lightness: -10%);   /* 10 % más oscuro */
}
.aviso {
  background: color.mix($marca, #ffffff, 15%);   /* 15 % de marca, 85 % de blanco */
}
.columna { width: math.div(100%, 3); }           /* división: 33.3333333333% */
```

También se pueden definir **funciones propias** con `@function` y `@return`:

```
@use "sass:math";

@function rem($px) {
  @return math.div($px, 16px) * 1rem;    /* convierte píxeles en rem */
}
h1 { font-size: rem(32px); }             /* compila a: font-size: 2rem; */
```

Mixin o función. El mixin **inserta declaraciones**; la función **devuelve un valor** que se usa dentro de una declaración.

8.6 Parciales y módulos

Sass permite dividir los estilos en varios archivos llamados **parciales**, cuyo nombre empieza por un guion bajo. Los parciales no se compilan por separado: se cargan desde un archivo principal con `@use`:

```
scss/
├── _variables.scss     colores, fuentes, puntos de ruptura
├── _mixins.scss
├── _base.scss          reset y etiquetas
├── _componentes.scss   botones, tarjetas
└── estilos.scss        archivo principal: solo carga los parciales
```

```scss
/* estilos.scss */
@use "variables" as v;     /* sin guion bajo ni extensión */
@use "base";
@use "componentes";

.titular { color: v.$color-marca; }    /* variable con el prefijo del módulo */
```

La antigua regla `@import` de Sass está **obsoleta** y se sustituye por `@use`, que carga cada módulo una sola vez y evita conflictos de nombres [25].

8.7 Otras características

- **`@extend` y marcadores (`%nombre`):** un selector hereda las reglas de otro.
- **Control de flujo:** `@if`, `@each` y `@for` generan reglas de forma automática.

```scss
$estados: (exito: #067647, alerta: #b54708, error: #b42318);

@each $nombre, $color in $estados {
  .aviso--#{$nombre} { border-left: 4px solid $color; }
}
/* genera .aviso--exito, .aviso--alerta y .aviso--error */
```

8.8 Compilación

El navegador **solo** carga archivos `.css`. El código Sass se compila de alguna de estas formas:

- **Línea de órdenes de Dart Sass:**
  - `sass scss/estilos.scss css/estilos.css` compila una vez;
  - con `--watch` vuelve a compilar cada vez que se guarda un cambio;
  - con `--style=compressed` genera un CSS sin espacios ni comentarios, más ligero, para publicar.
- **Extensiones del editor**, como las que compilan al guardar en Visual Studio Code.
- **Editores en línea:** en CodePen se elige SCSS como preprocesador del panel CSS y se puede ver el CSS compilado [26].
- **Herramientas de construcción** de proyectos, que compilan Sass junto con el resto del código.

Al compilar se puede generar un **mapa de código fuente** (*source map*, archivo `.css.map`). Con él, DevTools muestra en qué línea del `.scss` está cada regla, aunque el navegador use el `.css`.

Lo que se **valida** con el validador del W3C es el **CSS compilado**, no el SCSS.

Ideas clave. Un preprocesador (Sass, sintaxis SCSS) se compila a CSS estándar. Variables `$nombre` (se resuelven al compilar, a diferencia de las variables CSS), anidación con `&`, mixins (`@mixin`/`@include`, insertan declaraciones), funciones (`@function`/`@return` y módulos `sass:color`, `sass:math`, devuelven un valor), parciales `_archivo.scss` con `@use`. Se compila con `sass entrada.scss salida.css` y se valida el CSS resultante.

Referencias

[1] Mozilla. (s. f.). *MDN Web Docs: HTML y CSS*. https://developer.mozilla.org/es/

[2] WHATWG. (s. f.). *HTML Living Standard*. Web Hypertext Application Technology Working Group. https://html.spec.whatwg.org/multipage/

[3] W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. https://www.w3.org/TR/WCAG22/

[4] BEM. (s. f.). *BEM: Block Element Modifier*. https://getbem.com/

[5] W3C. (s. f.). *Selectors Level 4*. https://www.w3.org/TR/selectors-4/

[6] W3C. (s. f.). *CSS Cascading and Inheritance Level 4*. https://www.w3.org/TR/css-cascade-4/

[7] Gallagher, N. y Kerr, J. (s. f.). *Normalize.css*. https://necolas.github.io/normalize.css/

[8] W3C. (s. f.). *Media Queries Level 4*. https://www.w3.org/TR/mediaqueries-4/

[9] W3C. (s. f.). *Media Queries Level 5*. https://www.w3.org/TR/mediaqueries-5/

[10] W3C. (s. f.). *CSS Custom Properties for Cascading Variables Module Level 1*. https://www.w3.org/TR/css-variables-1/

[11] Meyer, E. A. y Weyl, E. (2023). *CSS: The Definitive Guide* (5.ª ed.). O'Reilly Media.

[12] Google. (s. f.). *Google Fonts*. https://fonts.google.com/

[13] W3C. (s. f.). *CSS Box Model Module Level 3*. https://www.w3.org/TR/css-box-3/

[14] W3C. (s. f.). *CSS Flexible Box Layout Module Level 1*. https://www.w3.org/TR/css-flexbox-1/

[15] W3C. (s. f.). *CSS Grid Layout Module Level 2*. https://www.w3.org/TR/css-grid-2/

[16] W3C. (s. f.). *CSS Validation Service*. https://jigsaw.w3.org/css-validator/

[17] W3C. (s. f.). *Nu Html Checker*. https://validator.w3.org/nu/

[18] Google. (s. f.). *Chrome DevTools*. Chrome for Developers. https://developer.chrome.com/docs/devtools

[19] Deveria, A. (s. f.). *Can I use: tablas de compatibilidad de HTML, CSS y JavaScript*. https://caniuse.com/

[20] WebAIM. (s. f.). *Contrast Checker*. Institute for Disability Research, Policy, and Practice, Utah State University. https://webaim.org/resources/contrastchecker/

[21] Marcotte, E. (2010). *Responsive Web Design*. A List Apart. https://alistapart.com/article/responsive-web-design/

[22] Wroblewski, L. (2011). *Mobile First*. A Book Apart.

[23] Bootstrap Team. (s. f.). *Bootstrap 5.3: documentación*. https://getbootstrap.com/docs/5.3/

[24] Tailwind Labs. (s. f.). *Tailwind CSS: documentación*. https://tailwindcss.com/docs

[25] Sass Team. (s. f.). *Sass: documentación*. https://sass-lang.com/documentation/

[26] CodePen. (s. f.). *CodePen: editor web de HTML, CSS y JavaScript*. https://codepen.io/